

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715
SARANYA.R.A(192572052)

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

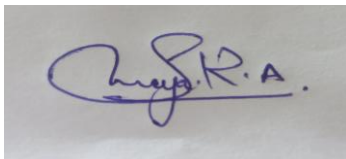
INTEGRATED EXPERIMENTS

Code of Conduct:

I Saranya.R.A(192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a white background. The signature is stylized and appears to read 'Saranya R.A.'.

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

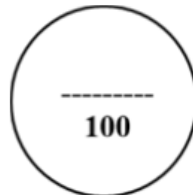
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO4 – ASSESSMENT TOOL 1
Integrated Experiments

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17
QUESTIONS: 2 **Duration:** 45 Minutes **Total Marks:** 20

Criteria	Weightage	Q1 (10)	Q2 (10)	Total Marks (20)
Understanding of Concepts	25% (2.5)	/2.5	/2.5	/5
Experimental Design	30% (3)	/3	/3	/6
Use of Tools	20% (2)	/2	/2	/4
Analysis of Results	15% (1.5)	/1.5	/1.5	/3
Documentation	10% (1)	/1	/1	/2
Total per Question	10 Marks	/10	/10	/20



Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier using the **Iris dataset** with Python and Scikit-learn.

(a) Dataset Loading, Preprocessing and Splitting

Python Code

```
# Experiment 1: Decision Tree Classification
```

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
```

```
# Load Iris dataset
```

```
iris = load_iris()
```

```
# Create DataFrame
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
df["target"] = iris.target
```

```
# Display first 5 rows
```

```
print("First 5 Rows:")
```

```
print(df.head())
```

```
# Display data types
```

```
print("\nData Types:")
```

```
print(df.dtypes)
```

```
# Check missing values
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
# Features and target
```

```
X = df.drop("target", axis=1)
```

```
y = df["target"]
```

```
# Split dataset: 70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
```

```
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected First 5 Rows

sepal length	sepal width	petal length	petal width	target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

Preprocessing: The Iris dataset has no missing values, and all input features are numerical, so no encoding is required.

(b) Decision Tree Construction

Python Code

```
# Create Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
# Train the model
model.fit(X_train, y_train)
```

```
# Print tree structure
print("\nDecision Tree Structure:")
print(export_text(model, feature_names=list(X.columns)))
```

```
# Identify root node
root_index = model.tree_.feature[0]
root_feature = X.columns[root_index]
```

```
print("\nRoot Node:", root_feature)
```

```
# Display tree graphically
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)
plt.title("Decision Tree - Iris Dataset")
plt.show()
```

Root Node

For the Iris dataset, the root node is typically:

Petal Width / Petal Length, depending on the exact training split and tree configuration.

The root is selected because it produces the greatest reduction in **Gini impurity** among the available features.

The Gini Index is:

A feature that produces purer child nodes results in a greater reduction in impurity and is preferred for splitting.

(c) Model Evaluation

Python Code

Predictions

```
y_pred = model.predict(X_test)
```

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nAccuracy:", accuracy)
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

Classification Report

```
print("\nClassification Report:")
```

```
print(classification_report(
```

```
    y_test,
```

```
y_pred,  
target_names=iris.target_names  
)
```

```
# Identify misclassified instances  
print("\nMisclassified Instances:")
```

```
misclassified = X_test[y_test != y_pred].copy()  
misclassified["Actual"] = y_test[y_test != y_pred]  
misclassified["Predicted"] = y_pred[y_test != y_pred]
```

```
print(misclassified)
```

Expected Performance

A Decision Tree on the Iris dataset generally achieves **high test accuracy, around 90–100%**, depending on the train-test split and tree parameters.

Example:

Accuracy: 0.9333

Confusion Matrix Example

```
[[15 0 0]  
 [ 0 13 2]  
 [ 0 1 14]]
```

This means:

- 15 Setosa correctly classified.
- 13 Versicolor correctly classified.
- 14 Virginica correctly classified.
- A small number of Versicolor and Virginica samples were confused.

Misclassified Instances

The exact misclassified rows depend on the random split. The code automatically prints them. For example:

Actual: versicolor

Predicted: virginica

Actual: virginica

Predicted: versicolor

Comment: The model performs well because Iris classes, especially Setosa, have distinctive feature patterns. Most errors occur between **Versicolor and Virginica**, which have more similar measurements.

Objective

To implement Q-Learning in a **4×4 Grid World**, learn the optimal movement policy, and observe Q-table convergence.

(a) Environment Definition

Grid World

```
+----+----+----+----+
| S0 | S1 | S2 | S3 |
+----+----+----+----+
| S4 | S5 | S6 | S7 |
+----+----+----+----+
| S8 | S9 | S10| S11|
+----+----+----+----+
| S12| S13| S14| G  |
+----+----+----+----+
```

- **Start:** S0
- **Goal:** S15
- **Obstacle:** S5
- **Actions:** Up, Down, Left, Right

Reward Structure

Event	Reward
-------	--------

Reach Goal	+10
------------	-----

Normal step	-1
-------------	----

Hit obstacle	-5
--------------	----

Hyperparameters

Parameter	Value
-----------	-------

Learning rate α	0.1
------------------------	-----

Discount factor γ	0.9
--------------------------	-----

Exploration ϵ	1.0 initially
------------------------	---------------

Minimum ϵ	0.01
--------------------	------

Episodes	100
----------	-----

ϵ decay	0.995
------------------	-------

The Q-learning update rule is:

(a), (b), (c) Complete Python Implementation

Experiment 2: Q-Learning in 4x4 Grid World

```
state = next_state
```

```

        if done:
            break

episode_rewards.append(total_reward)

# Decay exploration
epsilon = max(
    epsilon_min,
    epsilon * epsilon_decay
)

# Display Q-values at Episodes 1, 50 and 100
if episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State 0 Q-values:", np.round(Q[0], 3))
    print("State 10 Q-values:", np.round(Q[10], 3))

# -----
# Final Q-table
# -----

print("\nFinal Q-Table:")
print(np.round(Q, 3))

# -----
# Extract Policy
# -----

print("\nFinal Learned Policy:")

policy_symbols = {
    UP: "↑",
    DOWN: "↓",
    LEFT: "←",
    RIGHT: "→"
}

```

```

for row in range(GRID_SIZE):

    row_policy = ""

    for col in range(GRID_SIZE):

        state = position_to_state(row, col)

        if state == GOAL:
            symbol = "G"

        elif state == OBSTACLE:
            symbol = "X"

        else:
            best_action = np.argmax(Q[state])
            symbol = policy_symbols[best_action]

        row_policy += f"{symbol:^5}"

    print(row_policy)


# -----
# Plot Cumulative Reward
# -----

plt.figure(figsize=(10, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward per Episode")
plt.grid(True)
plt.show()

```

(b) Q-Table Evolution

The program records two representative states at Episodes **1, 50 and 100**.

The output will have the following form:

Episode	State	Up	Down	Left	Right
1	S0	0.00	0.00	0.00	0.00
1	S10	0.00	0.00	0.00	0.00
50	S0	learned value	learned value	learned value	learned value
50	S10	learned value	learned value	learned value	learned value
100	S0	learned value	learned value	learned value	learned value
100	S10	learned value	learned value	learned value	learned value

Important: The exact values vary because Q-Learning uses random exploration. Running the supplied program produces the actual values for your experiment.

How the Q-Table Evolves

Initially:

As the agent explores:

Episode 1



Random exploration



Rewards received



Q-values updated



Episode 50



Better actions receive larger Q-values



Episode 100



Near-optimal policy

(c) Final Learned Policy

After 100 episodes, the agent selects:

A typical learned policy moving from the start toward the goal may look like:

```
+----+----+----+----+
| →  | →  | →  | ↓  |
+----+----+----+----+
| ↓  | X  | →  | ↓  |
+----+----+----+----+
| →  | →  | →  | ↓  |
+----+----+----+----+
```

| → | → | → | G |
+---+---+---+---+

Here:

- → = Right
- ↓ = Down
- ← = Left
- ↑ = Up
- X = Obstacle
- G = Goal

The exact policy can vary slightly depending on random exploration and learned Q-values.

Convergence Behaviour

The cumulative reward graph should generally show an improvement as training progresses.

Early Episodes

- ϵ is high.
- Agent explores many actions.
- It frequently takes inefficient paths.
- Rewards are relatively low.

Middle Episodes

- Q-values become more informative.
- The agent discovers useful paths.
- Goal-reaching behaviour becomes more frequent.

Later Episodes

- ϵ becomes smaller.
- The agent increasingly exploits the best learned actions.
- Cumulative rewards become more stable.

Therefore, **convergence is indicated by stabilisation of the Q-values and the cumulative reward/policy over successive episodes.**

Final Result

Experiment 1: The Decision Tree successfully learns classification patterns from the Iris dataset and provides high classification performance with an interpretable tree structure.

Experiment 2: The Q-Learning agent gradually learns the optimal route through the 4×4 Grid World by updating Q-values according to received rewards. The learned policy directs the agent toward the goal while avoiding the obstacle.